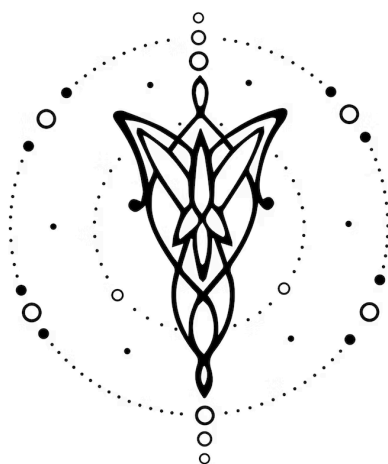




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

8 de marzo de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Jessica	Jones	10.001	jessica@itba.edu.ar
Luke	Cage	10.002	luke@itba.edu.ar
Danny	Rand	10.003	danny@itba.edu.ar
Matt	Murdock	10.004	matt@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/agustin-golmar/Flex-Bison-Compiler>.

3. Dominio

Desarrollar un lenguaje que permita crear y simular redes de comunicación con componentes básicos como *routers* y *hosts*. El lenguaje debe permitir enviar paquetes dentro de la red y entre sus componentes, configurar rutas, modificar los mensajes enviados e inspeccionar los paquetes a lo largo de su viaje por una red determinada.

Para reducir la complejidad del proyecto, los paquetes serán codificados como simples objetos con variables (similares a formato JSON¹), y se proveerán funciones básicas para operar y transformar dichos paquetes.

Toda la comunicación se realiza dentro de la memoria del mismo programa; las redes y componentes creados son completamente virtuales y solo existen dentro de la aplicación. Dicha comunicación se realiza sobre un protocolo ficticio, no existe noción real sobre TCP o UDP, así como tampoco un acceso real al hardware subyacente de la capa de red.

La implementación satisfactoria de este lenguaje permitiría experimentar diferentes arquitecturas de red, verificar sus ventajas, desventajas y potenciales vulnerabilidades antes de construir las mismas en un centro de cómputos o en un proveedor de infraestructura en la nube, minimizando los riesgos y costos finales.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán crear una o varias redes, y cada una tendrá nombre, *default gateway* y rango de direcciones en formato IPv4 (e.g., una red en el rango **10.1.0.0/16**).
- (II). Se podrán crear uno o varios *hosts* dentro de cada red, y cada uno tendrá un nombre y dirección IP asignada.
- (III). Se podrán agregar puertos de entrada y/o salida dentro de cada *host*.
- (IV). Se podrán crear paquetes para enviar a través de los puertos de un *host*, donde cada paquete contiene una o varias variables.
- (V). Las variables podrán ser de tipo *packet*, *address*, *integer*, *boolean* o *string*.
- (VI). Las variables podrán ser vectores de alguno de los tipos anteriores.
- (VII). Se proveerán operadores relacionales como $<$, $>$, $=$, $\neq$, $\leq$ y $\geq$.
- (VIII). Se proveerán operaciones aritméticas básicas como $+$, $-$, $*$ y $/$.
- (IX). Se proveerán operaciones lógicas básicas como AND, OR y NOT.
- (X). Se proveerán estructuras de control básicas de tipo IF-THEN-ELSE, FOR y WHILE.
- (XI). Se podrá definir un pequeño procedimiento sobre cada puerto dentro de un *host* que manipule los paquetes recibidos y/o emita otros en respuesta.
- (XII). Se podrán emitir paquetes desde un *host* hacia otro.
- (XIII). Se podrán crear tablas de ruteo dentro de cada *host*.

¹ <https://www.json.org/json-en.html>

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que comunique 3 redes a través de un *router*.
- (II). Un programa que construya 2 redes, una pública y otra privada.
- (III). Un programa que construya un 1 *host* dentro de una red.
- (IV). Un programa que construya y asigne 1 (una) tabla de ruteo.
- (V). Un programa que exponga un servicio en un *host*.
- (VI). Un programa que exponga un servicio en un *host*.
- (VII). Un programa que filtre paquetes en un *host*.
- (VIII). Un programa que implemente un balanceador de carga entre 2 *hosts*.
- (IX). Un programa que implemente un *proxy* entre 2 redes.
- (X). Un programa que manipule el contenido de los paquetes.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que asigne 2 servicios diferentes, en el mismo *host* y puerto.
- (III). Un programa que asigne un *host* a una red que no existe.
- (IV). Un programa que construya 2 redes con espacios de direcciones solapadas.
- (V). Un programa que intente operar entre tipos de datos incompatibles.

6. Ejemplos

En el **Cód. (6.1)**, se puede ver la creación de una red, con un *host* que ofrece una funcionalidad similar al protocolo ECHO en el puerto 7 (i.e., devuelve el mismo paquete que recibe).

```
// Crear una nueva red que soporte hasta 254 hosts:
network UnaRedDePrueba over 10.0.0.0/8;

// Crear un host dentro de la red anterior:
host UnHostConEcho in UnaRedDePrueba;

// Crear un servicio ECHO dentro del host:
add listening port 7 to UnHostConEcho {
    return packet;
}
```

Código 6.1: Implementación del protocolo ECHO.

En el **Cód. (6.2)**, se procede a crear un *firewall* que bloquee todos los paquetes que provienen desde el *host* con dirección IPv4 192.168.1.45, aunque permitiendo la propagación del resto.

```
// Crear un firewall dentro de la red del ejemplo anterior:
host UnFirewall in UnaRedDePrueba;

// Permitir que todos los paquetes pasen, excepto desde cierta IP:
add forwarding to UnFirewall {
  if (packet.source.ip = 192.168.1.45) {
    log "Se bloqueó el siguiente paquete desde {2}: {1}", packet, host.ip;
    drop;
  }
  else {
    return packet;
  }
}
```

Código 6.2: Implementación de un *firewall stateless* que bloquea cierta dirección IPv4.